TSOM: Time-Scoped Ontological Memory for LLM Agents

Project G1-08

Fatima Sohail (23I-0633)
Department of Computer Science
National University of Computer and Emerging Sciences
i230633@isb.nu.edu.pk

Menahil Ahmad (23I-0546)
Department of Computer Science
National University of Computer and Emerging Sciences
i230546@isb.nu.edu.pk

***Abstract*—Long-term memory lets LLM agents recall information across scenarios, but most memory systems, including A-Mem, keep old and new values of a changed fact side-by-side and have no notion of when a fact became false. We present TSOM (Time-Scoped Ontological Memory), an append-only fact layer that runs alongside an unchanged A-Mem. An update classifier decides whether each new fact adds to, re-affirms, replaces, or corrects what is stored, and a time-scoped retriever labels facts as [CURRENT] or [PAST] at question time. We compare TSOM with A-Mem and a RAG baseline using Qwen2.5-3B on FactConsolidation, 100 change scenarios and LoCoMo. On FactConsolidation SH 6K, TSOM raises accuracy from 40.0% to 67.0% and lowers the stale-answer rate from 65.8% to 41.9%, without reducing LoCoMo F1 (28.8 vs 28.7). However, it is the least accurate system on our change scenarios (73.0%), primarily because it retires values that should be kept, it does not improve past-period questions, and it triples the LLM calls per write. Failure analysis shows that most remaining stale answers occur with the correct value marked [CURRENT] in the context, so the model's own knowledge, not the memory, is the bottleneck.**

I. Introduction

LLM agents that interact with a user over many sessions need a long-term memory [1], because the context window is too small to hold the full conversation history. The usual approach is to store past conversations and retrieve the most relevant parts when a question arrives. The simplest version of this is retrieval-augmented generation (RAG) [2] over raw conversation turns. A-Mem [3] extends the idea; inspired by the Zettelkasten method, it converts each turn into a note with keywords, a short context and tags, links it to semantically similar notes, and updates the descriptions of older notes as new information comes in. This makes memory easier to search and organise, but it remains a collection of retrievable text.

The limitation is that the facts users share with an agent change over time. Suppose a user mentions in one session that they work at Amazon and, in a later session, that they now work at Microsoft. The second statement replaces the first, yet a text-based memory keeps both and leaves the model to work out which one is current. In A-Mem, the evolution step rewrites the context and tags of a note rather than the fact it contains, so neither note is marked as current or outdated. Time raises a similar problem. A question such as "Where did Ali work in 2018?" requires a different answer from "Where does Ali work now?", but these memories store only the date of the conversation, not the period during which the fact was true.

This paper addresses two gaps in current agent memory:

Gap 1 (update mechanism): Existing memories only append new information. Conflicting facts accumulate, and nothing records that one value replaced another, or that an earlier value was simply incorrect.

Gap 3 (time scoping): Existing memories have no notion of validity periods. They cannot represent that a fact held for a particular period, and they cannot restrict retrieval to the time period a question refers to.

We also considered a conflict gate that would skip A-Mem's evolution calls for unremarkable messages, running them only when a new fact looked like a correction or contradiction, or when a stored note was very similar to the new one. We postponed it to future work, because it would mean changing A-Mem itself, and TSOM is designed to run A-Mem unchanged. RQ3 therefore asks only how much extra cost the fact layer adds.

To address these gaps, we propose TSOM (Time-Scoped Ontological Memory). TSOM runs A-Mem unchanged and adds a second layer alongside it: an append-only store of facts, where each fact is an (entity, attribute, value) triple extracted from a conversation turn. We use the term "ontological" only in a lightweight sense, meaning that the memory organises itself and links its own relationships without a predefined schema. Attribute names are kept consistent at run time, and facts are connected to one another through links recording which fact replaced which.

For Gap 1, TSOM compares each new fact with what is already stored for the same entity and attribute, and then decides whether to add it, reaffirm it, replace the old value, correct it, or store it as information about an earlier period. Old facts are never deleted, only relabelled, so the full history can always be reconstructed. For Gap 3, TSOM keeps three kinds of time separate: when the system learned a fact, when the fact was true, and the tense of the wording. Validity dates are

filled in only when the text states them and are never guessed. At query time, TSOM determines whether a question concerns the present or the past and returns facts labelled [CURRENT] or [PAST] accordingly.

We study the following research questions.

- RQ1: Does tracking replacements and corrections of facts help an agent answer questions that depend on the latest value of a fact?
- RQ2: Do validity periods help an agent answer questions about when a fact was true?
- RQ3: How much additional cost, in LLM calls, latency and prompt tokens, does the fact layer introduce?

Our contributions are as follows.

1) We identify that text-retrieval memories such as A-Mem lack both an update mechanism and time scoping.
2) We present TSOM, a structured fact layer that sits alongside an unchanged A-Mem and stores self-organising, linked facts without ever deleting them.
3) We design a hybrid update classifier that handles clear cases with deterministic rules and defers the remaining cases to an LLM, together with a step that keeps attribute names consistent across turns.
4) We design a time-scoping method that separates learning time, validity period and wording tense, and never invents dates.

II. Related Work

A. Base Paper: A-Mem

A-Mem [3] turns each interaction into a note with keywords, tags and a contextual description, links it to similar notes, and lets new notes revise older ones. Its evolution step rewrites how a note is described, not the facts it states, so when two notes give different employers for the same person both remain and neither is marked as current, and nothing records when a statement was true. TSOM addresses these two gaps with a separate fact layer beside an unchanged A-Mem.

B. Memory Update and Conflict Handling

Bae et al. [4] framed memory updating as a task in which chatbots track the latest information about users with PASS, REPLACE, APPEND and DELETE operations. Mem0 chooses among ADD, UPDATE, DELETE and NOOP for each extracted fact, and its graph variant marks superseded relations as invalid [5]. Memory-R1 learns the same four operations with reinforcement learning [6], AriGraph deletes outdated triples [7], and HippoRAG 2 only appends passages [8]. TSOM differs in three ways. It never deletes or overwrites a fact, so the full history can always be rebuilt. Its operations (REPLACE, CORRECT, ADD_HISTORICAL and REAFFIRM) state whether a value changed, was wrong or describes an earlier period, which Mem0's operation set does not [5]. Finally, deterministic rules settle the clear cases before an LLM is asked, instead of a trained policy that needs supervision data [6].

C. Structured and Graph-Based Memory

Other work organises memory as layers or graphs. MemGPT [9] moves memory between tiers, HippoRAG 2 [8] and AriGraph [7] build graphs of extracted triples, and MemWeaver links graph, experience and passage layers back to the source dialogue [10]. HeLa-Mem consolidates episodic memories into general knowledge but notes that consolidation can add hallucinated facts [11], and LightMem defers contradiction resolution to an offline pass [12]. TSOM also stores triples, but beside A-Mem's notes and without a fixed ontology, and, given HeLa-Mem's warning, it treats a wrong supersession as costlier than a missed update.

D. Temporal Reasoning

Temporal knowledge has been studied for language models in general [13]. MemWeaver converts relative dates into absolute ones [10], and Zep [15] and APEX-MEM [14] attach validity intervals to facts. APEX-MEM, the closest work to ours, stores facts append-only in a property graph with a fixed 35-class ontology and agentic retrieval, at a high build cost [14]. TSOM shares the append-only and validity-interval ideas, but it decides at write time how each new fact relates to the old ones, uses no ontology or tool-using retrieval, and fills a validity period only when the text states it (Section IV-F). It therefore cannot place undated facts in time, a trade-off we accept to avoid inventing dates.

E. Benchmarks

MemoryAgentBench tests selective forgetting through its FactConsolidation task, on which all tested systems scored low for multi-hop questions and simple retrieval often beat specialised memory systems [16]. LongMemEval [17] also tests knowledge updates. We evaluate both gaps on FactConsolidation and on our own change scenarios, which add cancellations, additions and past-period questions, and use LoCoMo [18] only to check that general recall is not harmed.

III. Problem Formulation

This section defines the task that a memory system for a long-running LLM agent has to solve, and states how we split it into the two gaps from Section I.

A. Task Setting

The agent talks to a user over many sessions. The conversation arrives as a stream of turns T = (t_1, t_2, …, t_n), in time order. Each turn t_i has a speaker s_i, a text x_i, and the date d_i of the session it belongs to. After each turn, the memory is updated:

$$M_i = \mathrm{Write}(M_{i-1}, t_i)$$

After the full stream has been processed, the agent receives a question q. The memory returns a context C = Read(M_n, q), and the agent produces an answer from q and C. The agent, its prompts, the embedder and the number of retrieved items are the same for every memory design we compare, so the

only thing that differs between systems is what Write stores and what Read returns.

- Input: the turn stream T, and later a question q.
- Output: a context C that the agent uses to answer q, and the answer itself.

The memory is judged by whether the answer is correct, and by how much extra cost its Write and Read steps add.

B. Fact Representation

TSOM keeps a structured fact layer next to the note memory. A fact f is an (entity, attribute, value) triple, for example (Ali, works_at, Amazon), together with the following information.

- Learning time: the order in which the system learned the fact, and the session date of the turn it came from.
- Validity period: a start and an end date describing when the fact was true in the world. Either bound may be unknown.
- Tense: past, present or unknown, taken from the wording of the turn.
- Status: active, superseded or retracted.
- Links: references to the fact it replaced and the fact that replaced it.

Learning time, validity period and tense are kept separate on purpose. A user who mentions in 2023 that they work at Amazon has not told us that they started working there in 2023, so the session date must not be written into the validity period.

C. Gap 1: The Update Problem

When a new fact f_{new} arrives, let F be the set of active facts that share its entity and attribute. The update problem is to choose one operation for f_{new} given F.

- ADD: f_{new} is the first value, or it can hold together with the existing values.
- REAFFIRM: f_{new} repeats an existing value.
- REPLACE: the value has changed over time, so the old fact becomes superseded.
- CORRECT: the old value was wrong, so the old fact becomes retracted.
- ADD_HISTORICAL: f_{new} describes an earlier period, so it is stored as already superseded and the current fact is left alone.

If the decision cannot be determined (for example, because the LLM reply is unreadable), the memory stays unchanged.

The goal is that, for any entity and attribute, the memory can say which value holds now, which values held before, and which were mistakes. A memory that only appends corresponds to always choosing ADD, which is why it cannot answer this.

Wrong supersession is treated as more costly than a missed update. Wrongly replacing a true fact loses information, while missing an update only leaves an extra value in memory.

D. Gap 3: The Time Scoping Problem

Given a question q, the memory first decides whether q is about the present or about a past period. If q names a specific year or month, that date becomes the target date τ. The memory then has to return facts that fit that time.

- For a current question, return only active facts.
- For a historical question, return the facts whose validity period does not provably exclude τ. If a bound is unknown, it is treated as open.

Each returned fact is labelled [CURRENT] or [PAST], so the agent can see which values still hold. The result is that "Where does Ali work now?" and "Where did Ali work in 2018?" lead to different contexts built from the same memory.

E. Constraints and Assumptions

The design follows these rules.

- Append-only: facts are never deleted. Only status, end date, replacement link and observation count may change, so the full history can always be reconstructed.
- No invented dates: validity dates are filled in only when the text states them. If the order of two dates cannot be established, it is treated as unknown.
- A-Mem is run unchanged, and the fact layer is added beside it.

The formulation also assumes the following simplifications. Values are matched by normalised exact string, so "NYC" and "New York" are treated as different values. Entities are matched by name, without coreference resolution. Time scoping depends on dates being stated in the text, and without them the memory can only fall back on the order in which facts were learned.

F. Research Questions and Hypotheses

Each hypothesis answers the matching research question in Section I.

- H1 (RQ1): An agent using TSOM answers questions that depend on the latest value of a fact more accurately than an agent using A-Mem or RAG, because TSOM marks which value is current instead of leaving both in the context.
- H2 (RQ2): TSOM answers questions about a specific past period more accurately than A-Mem or RAG, because it can filter facts by validity period and label them as past.
- H3 (RQ3): The fact layer adds extra LLM calls per write, since it extracts facts and classifies updates, but the deterministic rules reduce how many of those updates need an LLM.

IV. Proposed System: TSOM

A. Overview

TSOM has a write path and a read path, shown in Fig. 1. On the write path, every conversation turn is first passed to A-Mem, which creates and links a note as usual. The same turn is then sent to the fact layer, which extracts facts, decides how each one relates to what is already known, and stores the result. On the read path, A-Mem retrieves its top-k notes for

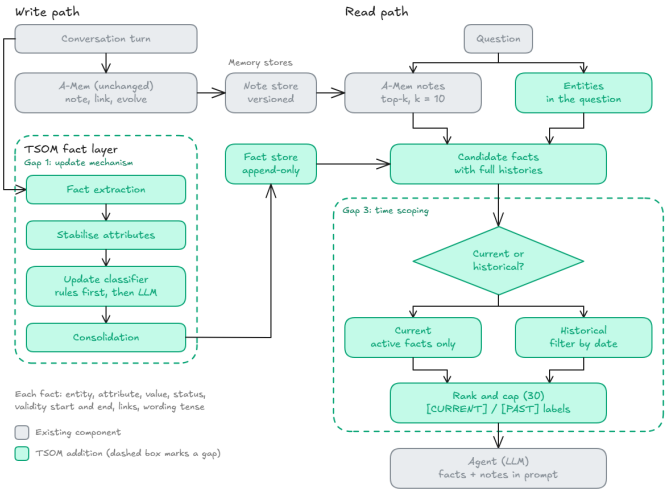


Fig. 1. Architecture of TSOM.

the question, and TSOM adds the relevant facts from the fact store, each labelled [CURRENT] or [PAST]. The answering agent receives both. It uses the same prompts, embedder and retrieval depth as the baselines, so the only difference between systems is what the memory returns.

B. Components and Roles

Several components in TSOM are backed by an LLM, and each has a single narrow job.

- A-Mem (unchanged): creates notes with keywords, context and tags, links similar notes, and evolves older notes. It provides the text-retrieval half of the memory.
- Fact Extractor (LLM): reads a turn and outputs atomic (entity, attribute, value) triples, any dates the text states explicitly, and the tense of the wording.
- Attribute Stabiliser (embedder and string rules): keeps relation names consistent across turns.
- Update Classifier (rules first, LLM second, or LLM only as an ablation): decides how a new fact relates to the existing facts for the same entity and attribute.
- Consolidator: applies the decision to the fact store.
- Time-Scoped Retriever (embedder and pattern rules): selects, filters, ranks and labels facts at question time.
- Answering agent: the same agent used for all systems. It is not part of TSOM and only reads the context.

C. Tools and State

The agent has two memory tools: write_memory stores a conversation turn, and search_memory takes a query and returns the top-k notes together with the labelled facts. The tsom_history variant adds a third tool, get_fact_history, which lists everything memory holds about an entity, past values included. The agent runs one fixed loop: for each conversation turn it calls write_memory, and for each question it asks the LLM for search keywords, calls search_memory with them (k = 10) and answers from the returned context with A-Mem's answer prompts. In tsom_history, the LLM first decides whether to call get_fact_history and for which entity, and the returned history is placed before the retrieved context. The tools run on the following software. The LLM runs locally through Ollama, with qwen2.5:3b [19] as the default model. The sentence embedder is all-MiniLM-L6-v2 [20], shared with the baselines. Facts are stored in a SQLite database.

The state consists of three parts. The first is the A-Mem note store, which TSOM does not modify. The second is the fact store. Each fact holds its entity, attribute and value, the order in which it was learned, the session date, a validity start and end (filled only if the text states them), the tense, a status (active, superseded or retracted), links to the facts it replaces and the fact that replaced it, and a count of how often it was reaffirmed. The third is a set of note snapshots, described in Section IV-F.

The fact store is append-only. Facts are never deleted or overwritten. Only the status, end date, replacement link and observation count of an existing fact may change, so the full history can always be reconstructed.

D. Fact Extraction and Attribute Stabilisation

The Fact Extractor receives the turn text, with A-Mem's "Speaker X says:" wrapper removed, together with the speaker's name so that "I" and "my" resolve to the right person. It returns triples, any dates that are explicitly stated, and the tense of the wording. Dates are never guessed. For numbered fact lists, where many short statements arrive together, facts are extracted in batches of 10 statements per call, with each fact tied to its source statement.

Updates can only be detected if the same relation gets the same name every time. If one turn produces speaks and another produces speaks_language, a contradiction between them goes unnoticed. The Attribute Stabiliser addresses this in two ways. First, the extractor is shown the attribute names already in use, preferring those of entities mentioned in the current turn. Second, after extraction, a new attribute name is merged into an existing one for the same entity only if the two match after removing filler words and plurals, or if the cosine similarity of their embeddings is at least 0.88. The threshold is deliberately strict. Related but different relations such as speaks and speaks_to score around 0.82, and merging them would wrongly overwrite a true fact. The original name is kept for audit.

This step is what makes the memory self-organising. The relation vocabulary is not fixed in advance but grows and stays consistent as conversations are processed.

E. Gap 1: Update Mechanism

The Update Classifier compares each new fact with the active facts for the same entity and attribute, and assigns one of the following operations.

- ADD: the first value, or a value that can hold alongside the existing ones.
- REAFFIRM: the same value again. The old fact stays active and its count increases by one.

- REPLACE: the value changed over time. The old fact is marked superseded and linked to the new one.
- CORRECT: the old value was wrong. The old fact is marked retracted and linked to the new one.
- ADD_HISTORICAL: new information about an earlier period. The new fact is stored as already superseded and the current fact is untouched.
- UNRESOLVED: the LLM reply could not be read, and nothing changes.

The classifier is hybrid. Deterministic rules settle the clear cases without calling a model.

- If there is no existing fact, the operation is ADD.
- If the value is identical to an existing one, it is REAFFIRM.
- If the new fact's stated start date provably precedes all existing ones, it is ADD_HISTORICAL.
- If the wording is past tense and the stated start precedes the existing fact's known start, it is ADD_HISTORICAL.
- If the key already holds two or more active values, it is treated as multi-valued and the operation is ADD.

Everything left over is sent to the LLM, which chooses between REPLACE, CORRECT and ADD at temperature 0 and gives a short reason. Two optional rules exist and are off by default: one adds a value directly when the attribute is on a list of multi-valued relations (such as languages or hobbies), and one adds a value when the sentence contains "also" or "too" without a change word such as "now" or "moved".

The Consolidator then applies the decision as a new row, relabels the old row, and records links in both directions. Each fact is applied inside a database transaction. If it fails, the transaction is rolled back, the failure is counted, and the A-Mem note is kept. A guard also reroutes any REPLACE that would end a fact before it began into ADD_HISTORICAL.

The design assumes that wrongly replacing a true fact is more costly than missing an update. For this reason the thresholds are strict, and an unreadable LLM reply leaves memory unchanged.

F. Gap 3: Time Scoping

TSOM keeps three notions of time apart. Learning time is when the system learns the fact. The validity period is when the fact was true in the world, and it is filled in only when the text states it. Tense is a hint from the wording and never writes into the validity dates, since "used to live in Paris" says nothing about when.

TSOM places facts in time without inventing dates. When a fact is replaced, its end date becomes the start date stated for the replacement. If no start is stated, the end stays empty. Session dates are never written into validity fields. Dates are compared only when both parse cleanly (year, year-month or full date) and at the coarser precision, so "2020" is not treated as earlier than "2020-05". A missing or unparseable date means the order is unknown. At question time only, the session date of a replacement may be used to bound the replaced fact, since it ended no later than that session. This is computed on the fly and not stored.

The Time-Scoped Retriever works in the following steps.

1) A pattern-based check classifies the question as current or historical. A year, a month name, or a phrase such as "used to", "previously" or "back in" marks it as historical, and a named year or month becomes the target date.
2) For a current question, only active facts are shown.
3) For a historical question, each non-retracted fact gets an estimated interval from its stated dates or its replacement's start. A fact is shown unless the target date lies provably outside its interval, and unknown bounds count as open. If no fact has usable bounds, the full history is shown so the model can reason from the ordering.
4) Candidate facts come from the notes A-Mem retrieved, expanded to the full history of each relation, plus all facts of any entity named in the question, which helps multi-hop questions.
5) At most 30 fact lines are returned. Relations are ranked by embedding similarity to the question, entities named in the question win ties, and when a relation is truncated its newest lines are kept.
6) Facts are presented in a fixed format, for example "[CURRENT] Ali | works_at | Microsoft" or "[PAST] Ali | works_at | Amazon (replaced at step n)", and placed before the retrieved notes.

A-Mem silently rewrites the context, tags and links of old notes. Whenever this happens, TSOM saves a snapshot of the previous state and records which new note caused the change, so changes to notes can also be traced.

V. Experimental Setup

A. Datasets

We use three datasets, all for evaluation only. No model is trained, so there are no training, validation or test splits. The one separation we keep is on LoCoMo, where one conversation was used for development and is excluded from evaluation.

The main benchmark is FactConsolidation, taken from the Conflict_Resolution part of MemoryAgentBench [16]. It presents a long stream of numbered facts in which some facts are later changed, for example a player who plays goalkeeper and is later said to play flanker. Questions ask for the newest value. Single-hop (SH) questions depend on one fact, while multi-hop (MH) questions chain several facts and are much harder. We use both variants at stream lengths of 6K and 32K, with 100 questions in each setting. Facts are written to memory in chunks of 20 numbered facts. This dataset is our main probe for Gap 1 (update mechanism), because its questions always ask for the newest value. We also run the Gap 3 ablation on it, to check how the system behaves when old values are never retired.

The second dataset is our own set of change scenarios, built to cover cases that FactConsolidation lacks, such as additions and cancellations. Each scenario is a short dialogue about a personal fact, with a few filler sessions in between, and comes with hand-labelled facts and a question. The scenario file holds 210 hand-reviewed scenarios, and we evaluate on a balanced

subset of 100 drawn from four kinds: change, cancel, additive and restate. An answer is scored as correct if it contains the current value and avoids the old one. Table 1 gives one example of each kind. Change and cancel scenarios also carry a past-period question, which makes this dataset our main probe for Gap 3 (time scoping). Both gaps are evaluated on both datasets through the ablations described in Section V-C. A hand-labelled set of 200 facts from these scenarios is used to measure extraction recall.

The third dataset is LoCoMo [18], which has long conversations with questions in five categories. We use it only as a no-harm check, to see whether the fact layer reduces general recall, and not as a headline result. We evaluate on test conversations 1 and 8, which have 369 and 509 turns and 301 questions in total. Conversation 0 was the development set and is excluded.

B. Systems Compared

All systems share the same agent, tools, answer prompt and embedder (all-MiniLM-L6-v2 [20]), so the only component that changes is the memory module. We compare six systems.

rag is the baseline. It stores raw conversation turns and retrieves the most similar ones, and it makes no LLM calls when writing. amem is the base paper, using A-Mem's own released code unchanged and run under our settings. tsom is our full system, which runs A-Mem and adds the fact layer, made up of an extractor, an update classifier (Gap 1) and a time-scoped retriever (Gap 3). The remaining three are variants of TSOM. gap3_only switches the classifier off, so every new value is simply added and nothing is retired. gap1_only switches the time-scoped retriever off, so all active facts are shown as [CURRENT] with no handling of past periods. tsom_history is TSOM with an optional get_fact_history tool that the agent may choose to call.

C. Experiments and Job Design

The runs are organised into six experiments, E1 to E6, listed in Table 2. In total we ran 38 jobs, where each job is one run of one memory system on one dataset. Table 3 shows how the jobs are split across datasets.

D. Models and Settings

All systems use one language model, qwen2.5:3b [19] served through Ollama on NVIDIA Tesla T4 GPUs (Google Colab, and Kaggle for one of four workers). The same model handles every LLM call: fact extraction, update classification, A-Mem note construction and evolution, keyword generation and answering. The embedder is all-MiniLM-L6-v2 [20], shared by RAG and A-Mem, and all configurations use a retrieval depth of k = 10.

The context window is set to 16384 tokens. We raised it from the Ollama default of 4096 because the default silently truncated long prompts. Every call runs at temperature 0, including A-Mem's internal calls, and the seed is sent to the model. This overrides the temperatures in the original code, which are 0.7 for most answers and 0.5 for adversarial questions. Output is capped at 1000 tokens and failed calls are retried twice.

Most jobs use one seed. The exception is FactConsolidation 6K, where rag, amem and tsom were run with seeds 1, 2 and 3 to test whether temperature 0 makes runs repeatable. Because a single seed was used elsewhere, results at 32K, on the scenarios, on LoCoMo and in the ablations should be read as single runs. Finally, the FactConsolidation prompt is the same for every system but tells the model to prefer the larger-numbered fact or the one marked CURRENT. Only TSOM-based systems show CURRENT markers, so this prompt may favour them, and we discuss this in Section X.

E. Implementation Details

The code is written in Python and uses Ollama as the model backend. Experiments ran on Tesla T4 GPUs on Google Colab and Kaggle, split across four workers. Each run is defined by a configuration file, and each run saves per-question predictions, a per-operation log of LLM calls, tokens and latency, the final SQLite memory, and summary tables. In addition to the main jobs, we ran three further checks. An extraction job measures the fraction of the 200 labelled facts that the extractor finds. A repeatability check compares answers across the three repeated FactConsolidation 6K runs; between 54% and 100% of answers were identical across seeds, so temperature 0 did not make the runs fully deterministic (see Section X). A failure analysis sorts every wrong TSOM answer on FactConsolidation into a category by checking the fact store and the memory shown to the model. We also record step-by-step traces of two successes and two failures each on FactConsolidation and the change scenarios. The metrics, the confidence intervals and the way A-Mem is run are described in Section VI.

VI. Evaluation Methodology

All systems share the agent, tools and answer prompt described in Section V, so differences between rows come from the memory module alone.

Update correctness. On FactConsolidation, predictions and gold answers are normalised as in SQuAD [21], and an answer is correct if one contains the other. A wrong answer is stale if it matches an earlier value of the same fact, recovered from facts with identical wording around the answer. Stale rates are computed only over questions whose fact changed (n = 74, 64, 80 and 82 for SH 6K, SH 32K, MH 6K and MH 32K). For MH questions only the last fact in the chain is checked, so these rates are lower bounds. The active contradiction rate counts, after each session, the (entity, attribute) keys holding more than one active value, per 100 active facts. Since A-Mem stores no structured facts, we compare against the same extracted facts with nothing retired, which is what an append-only memory would hold.

Change scenarios. An answer is correct if it contains the current value (all values for additive scenarios), stale if it contains only old values, and hedged if it contains both. Past-period questions (n = 48) test whether the earlier value can

Kind	Example dialogue	Labelled facts	Question and scoring	What it tests
Change (a value is replaced)	Ali, 8 May 2023: has lived in Karachi for three years. Ali, 14 Jun 2023: moved to Lahore last month.	lives_in = Karachi, then lives_in = Lahore (replaces Karachi)	"Where does Ali live now?" Correct: Lahore. Stale: Karachi. Past-period question "Where did Ali live before?" Correct: Karachi.	The old value must be retired, but stay retrievable when asked about the past.
Cancel (a value is withdrawn)	Fatima, 5 Jun 2023: booked a flight to Dubai. Fatima, 10 Aug 2023: cancelled the flight.	has_booking = flight to Dubai, then has_booking = cancelled	"What is the status of Fatima's flight to Dubai?" Correct: cancelled or called off. Stale: still booked. Past-period question "What did Fatima have booked in June 2023?" Correct: flight to Dubai.	No new value replaces the old one, so the system must recognise a cancellation.
Additive (a second value, nothing retired)	Ayesha, 12 Jul 2023: has a dog called Max. Ayesha, 2 Aug 2023: also got a parrot.	has_pet = dog, then has_pet = parrot (both kept)	"What pets does Ayesha have?" Correct: dog and parrot. No past-period question.	The system must not over-retire. A classifier that treats every new value as a replacement would drop the dog.
Restate (the same value again)	Usman, 5 Sep 2023: is a primary school teacher. Usman, 30 Sep 2023: being a teacher is still the best job.	job_title = teacher, then job_title = teacher (redundant)	"What is Usman's job now?" Correct: teacher. No past-period question.	A repeated value must not be treated as a change or lose the stored fact.

TABLE 1
SCENARIO KINDS WITH ONE EXAMPLE EACH.

Experiment	Systems	Purpose
E1 Baseline	rag	Measure how well plain retrieval of raw conversation turns performs, which gives the floor for all other systems.
E2 Base paper	amem	Measure A-Mem under the same settings as every other system, by running its released code on our datasets.
E3 Full system	tsom on FactConsolidation and the change scenarios; tsom_history on the change scenarios	Test whether the full TSOM system corrects outdated and contradicted facts, and whether the optional get_fact_history tool changes the outcome.
E4 Gap-wise evaluation	gap1_only and gap3_only, compared with tsom and amem (FactConsolidation 6K and the change scenarios)	Test whether the component for each gap helps on its own: the update classifier for Gap 1 and the time-scoped retriever for Gap 3.
E5 Ablation	Same runs as E4, read as the full system minus one component	Show how much each component adds to the full system.
E6 Efficiency	All runs	Measure the cost of the fact layer using LLM calls, tokens, and p50 and p95 latency per write, plus reliability counts (unparseable classifier replies and failed fact steps).

TABLE 2
EXPERIMENTS E1 TO E6.

still be recovered. Extraction recall is the share of the 200 hand-labelled facts found with the right entity and value.

General recall and cost. LoCoMo answers are scored with A-Mem's own token-level F1 code, unchanged, so our numbers stay comparable with our runs of A-Mem. For every write we log LLM calls, tokens and latency; on FactConsolidation a write is a chunk of 20 facts, on LoCoMo one turn.

Statistics and A-Mem runs. Each result is the mean over seeds ($\pm$ standard deviation where three seeds exist) with a 95% bootstrap confidence interval over questions or scenarios (1,000 resamples) [22]. We call a difference significant only when the intervals do not overlap. All A-Mem results come from its released code, run unchanged with our model, 16K context and temperature 0.

VII. RESULTS

A. Gaps 1 and 3: outdated and contradicted facts

On SH 6K, TSOM raises accuracy from 40.0% (A-Mem) to 67.0% and lowers the stale-answer rate from 65.8% to 41.9% (Table 4), the only stale-rate difference whose intervals do not overlap. SH 32K points the same way, while multi-hop accuracy stays low for every system. TSOM ends every run with no active contradictions, against 23.0–24.9 per 100 facts when nothing is retired (Fig. 2); this shows that old values are retired consistently, not that answers improve.

The change scenarios tell a different story (Table 5). Stale answers are rare for every system, and TSOM is the least accurate overall, although its interval overlaps A-Mem's. It is marginally ahead on change scenarios but loses on the kinds where nothing should be retired, most clearly additive and

Dataset	Systems	Seeds	Jobs
Extraction check (200 labelled facts)	extractor only	1	1
FactConsolidation 6K, single-hop and multi-hop	rag, amem, tsom	3	18
FactConsolidation 6K, single-hop and multi-hop	gap1_only, gap3_only	1	4
FactConsolidation 32K, single-hop and multi-hop	rag, amem, tsom	1	6
Change scenarios (100)	rag, amem, tsom, tsom_history	1	4
Change scenarios (100)	gap1_only, gap3_only	1	2
LoCoMo, conversations 1 and 8	rag, amem, tsom	1	3
Total			38

TABLE 3
JOBS BY DATASET.

Split	System	Accuracy (%)	Stale (%)	Contr.
SH 6K	RAG	38.0 [29.0, 48.0]	65.8 [55.0, 76.6]	–
SH 6K	A-Mem	40.0 [30.7, 50.0]	65.8 [55.9, 76.1]	–
SH 6K	TSOM	67.0 [58.0, 76.0]	41.9 [31.1, 54.1]	0.0 vs 24.6
SH 32K	RAG	38.0 [28.0, 48.0]	53.1 [40.6, 65.6]	–
SH 32K	A-Mem	42.0 [32.0, 51.0]	45.3 [32.8, 57.8]	–
SH 32K	TSOM	79.0 [71.0, 87.0]	31.2 [20.3, 42.2]	0.0 vs 23.0
MH 6K	RAG	3.0 [0.0, 7.0]	15.0 [7.5, 22.5]	–
MH 6K	A-Mem	5.3 [1.7, 9.7]	17.9 [10.0, 26.2]	–
MH 6K	TSOM	16.3 [10.0, 23.3]	12.5 [5.8, 20.0]	0.0 vs 24.9
MH 32K	RAG	2.0 [0.0, 5.0]	28.0 [18.3, 37.8]	–
MH 32K	A-Mem	2.0 [0.0, 5.0]	19.5 [11.0, 28.0]	–
MH 32K	TSOM	6.0 [2.0, 11.0]	24.4 [15.9, 34.1]	0.0 vs 23.0

TABLE 4
FACTCONSOLIDATION ACCURACY AND STALE-ANSWER RATE (%) WITH 95% BOOTSTRAP CONFIDENCE INTERVALS. 6K: MEAN OF THREE SEEDS (STANDARD DEVIATION ACROSS SEEDS $\leq$ 2.3 POINTS; PER-CELL VALUES IN RESULTS/PHASE2/TABLES.MD); 32K: ONE SEED. CONTR. = ACTIVE CONTRADICTIONS PER 100 FACTS FOR TSOM, AGAINST THE SAME FACTS WITH NOTHING RETIRED.

System	Correct (%)	Stale (%)	Past-period (%)
RAG	90.0 [84, 96]	6.0 [2, 11]	93.8 [85.4, 100]
A-Mem	82.0 [74, 89]	5.0 [1, 10]	97.9 [93.8, 100]
TSOM	73.0 [65, 82]	4.0 [1, 8]	91.7 [81.2, 97.9]

TABLE 5
CHANGE SCENARIOS (SEED 1): OVERALL CORRECT AND STALE ANSWERS, AND PAST-PERIOD ACCURACY (%, 95% CI).

cancel (Fig. 3). For Gap 3, every system answers over 90% of past-period questions, and TSOM does not improve on A-Mem.

B. General recall on LoCoMo

Overall LoCoMo F1 is unchanged (Table 6), so the fact layer does not harm general recall. Temporal and multi-hop F1 rise while adversarial F1 falls; we suspect a confident [CURRENT] fact makes the model less willing to say that

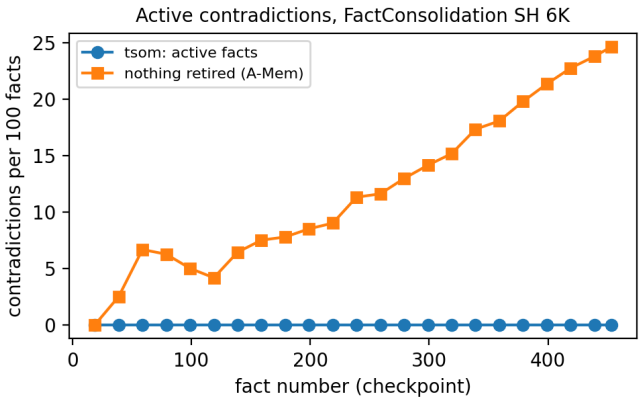


Fig. 2. Active contradictions per 100 facts at each session checkpoint on FactConsolidation SH 6K (seed 1). TSOM's active facts are compared with the same facts when nothing is retired.

Category (n)	RAG	A-Mem (Ours)	TSOM
Single-hop (117)	22.4	25.3	25.8
Multi-hop (48)	16.6	17.7	24.1
Temporal (59)	30.5	31.4	33.2
Open-domain (13)	18.2	25.2	22.3
Adversarial (64)	36.5	41.3	35.1
Overall (301)	25.9	28.7	28.8

TABLE 6
LOCOMO F1 ON TEST CONVERSATIONS 1 AND 8 (SEED 1).

information is missing, but we did not test this, and with two conversations and one seed these differences are indicative only. We do not compare with the LoCoMo scores published for A-Mem, which cover all ten conversations at a different temperature; all A-Mem numbers here come from its released code run under our settings.

C. Gap-resolution table

Against the hypotheses in Section III-F (Table 7), H1 is only partly supported: TSOM beats both baselines on FactConsolidation but is the least accurate on the change scenarios, a surprising result since these scenarios target exactly the job of the fact layer. H2 is not supported, since TSOM does not improve past-period questions. The first part of H3 holds, as the fact layer adds LLM calls to every write (Section VIII-B); the second part, that the rules reduce how many updates need an LLM, was not tested because we did not run the llm_only classifier.

VIII. Ablation and Comparative Analysis

A. What each component contributes

We ran two reduced versions of TSOM (seed 1): gap3_only turns the update classifier off, so nothing is retired, and gap1_only shows all active facts as [CURRENT]. A component's effect is the full system minus the run without it (Fig. 4).

Update classifier (Gap 1). The classifier accounts for most of the FactConsolidation gain (+16.0 points of SH 6K

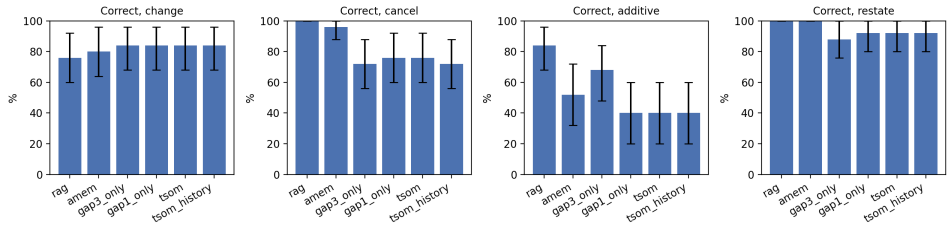


Fig. 3. Correct answers per scenario kind (n = 25 each, seed 1, 95% CIs).

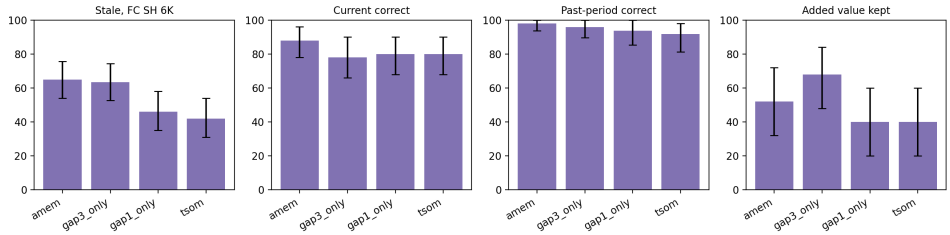


Fig. 4. Ablation (seed 1, 95% CIs): SH 6K stale rate, current and past-period accuracy, and added values kept.

Gap	Metric	A-Mem	TSOM	Addressed?
1+3 (fact layer)	Stale rate, FC SH 6K	65.8%	41.9%	Partially
1+3 (fact layer)	Accuracy, change scenarios	82.0%	73.0%	Not
1 (classifier)	Contradictions per 100 facts, SH 6K	24.6†	0.0	Fully
3 (retriever)	Past-period correct	97.9%	91.7%	Not
Write cost (secondary)	LLM calls per write, FC SH 6K	2.91	9.70	Measured only

† Same extracted facts with nothing retired.

TABLE 7
GAP-RESOLUTION TABLE. A-MEM IS OUR REPRODUCTION UNDER IDENTICAL SETTINGS.

Data	System	LLM calls	Tokens	p50 (s)	p95 (s)
FC SH 6K	A-Mem	2.91	10,051	9.96	21.22
FC SH 6K	gap3_only	4.91	12,139	21.84	33.26
FC SH 6K	gap1_only	9.65	14,075	26.42	38.47
FC SH 6K	TSOM	9.70	13,633	25.75	37.22
LoCoMo	A-Mem	2.76	1,864	4.06	5.69
LoCoMo	TSOM	3.90	2,415	4.13	6.51

TABLE 8
COST PER WRITE (SEED 1). RAG MAKES NO LLM CALLS (0.01 S PER WRITE).

accuracy, −21.6 points of stale rate and +10.0 points of MH 6K accuracy). It also causes the largest regression: both values were kept in 28.0 points fewer additive scenarios, because the model often classifies a second value as a replacement when the multi-valued and "also" rules are switched off. Even without the classifier, the fact layer lifts SH 6K accuracy from 40.0% to 51.0%.

Time-scoped retriever (Gap 3). The retriever adds 5.0 points of SH 6K accuracy but lowers past-period accuracy by 2.1 points. Without stated dates it can only act on values the classifier has replaced, and our scenarios rarely state dates, so its effect cannot be separated from Gap 1 on this data. The optional get_fact_history tool also made no measurable difference.

B. *Efficiency and cost*

On FactConsolidation, where a write holds 20 facts, TSOM makes 3.3 times as many LLM calls per write as A-Mem and more than doubles median write latency (Table 8). The gap3_only run shows that most of the extra calls come from the classifier, while the retriever adds almost nothing to write cost. On LoCoMo, where most turns carry no factual update, the overhead is small. No classifier reply was unparseable and no fact step failed in any run.

IX. FAILURE ANALYSIS

We assigned every wrong TSOM answer on FactConsolidation (800 questions across splits and seeds) to one of five categories (Table 9) and traced two successes and two failures through the agent's steps (Table 10). In both successes the classifier replaced the old value and only the newest one reached the model as [CURRENT]. F1 follows the same path, yet the model still gave the old value, while F2 fails when the fact is stored.

The model overrides the memory (14.1%). Of the 133 stale answers, 113 (85%) were given with the correct value marked [CURRENT], as in F1. FactConsolidation's counterfactual edits [23] often contradict the model's own knowledge (the Salem-Keizer Volcanoes are a real baseball team), and models are known to prefer that knowledge over their context [24], [25]. Only one stale answer came from the store keeping an

Category	Count	Share of questions
Wrong, not stale	242	30.2%
Stale, correct value shown as current	113	14.1%
Newest value never stored	90	11.2%
Stale, correct value not in context	19	2.4%
Stale, store kept the old value	1	0.1%

TABLE 9
FAILURE CATEGORIES FOR TSOM ON FACTCONSOLIDATION (800 ANSWERS).

old value. Checking the answer against the [CURRENT] fact, or a larger model, may help.

Extraction misses and aliases (11.2% and 2.4%). In 11.2% of questions the newest value was never stored, consistent with an extraction recall of 45.5%. In F2, London was stored under other names for the country, such as *Great Britain*, so *Rupnagar* for *United Kingdom* was added instead of replacing it, because entities are matched by name only (Section III-E). In a further 2.4% the current value was stored but not retrieved. A stronger extractor, alias resolution and entity-based fact lookup would address these cases.

Wrong but not stale (30.2%). Most of these answers come from multi-hop questions, where the difficulty is composing facts rather than keeping them current; one search per hop is a natural next step.

Additive values (change scenarios). The classifier often retires the first value when a second one is added (Section VIII-A). TSOM already has rules for this case (a list of multi-valued attributes and an "also" cue), but they were switched off in our runs, so enabling them is the obvious fix.

X. LIMITATIONS AND THREATS TO VALIDITY

Model and prompt. We used only Qwen2.5-3B [19], and larger models may follow a [CURRENT] fact more closely. The FactConsolidation answer prompt tells every system to prefer the larger-numbered fact or the one marked CURRENT, but only TSOM shows such markers, so part of its gain may come from this instruction; on the change scenarios, whose prompt lacks it, TSOM shows no gain.

Sample size and repeatability. LoCoMo covers two conversations and most runs use one seed, so many intervals overlap. Even at temperature 0, only 54–100% of answers were identical across seeds.

Data and bias. FactConsolidation's counterfactual edits exaggerate the conflict between memory and the model's knowledge, and its stale labels are recovered heuristically. Our scenarios rarely state dates, so they test the retriever only weakly, and they were written by the system's authors, although prompts and rules were developed only on the excluded LoCoMo conversation 0. We compare against one memory architecture and one retrieval baseline, so the conclusions may not transfer to other designs.

XI. CONCLUSION

We presented TSOM, an append-only fact layer beside an unchanged A-Mem, with an update classifier that retires outdated facts and a time-scoped retriever that labels facts [CURRENT] or [PAST]. On FactConsolidation SH 6K it raised accuracy from 40.0% to 67.0% and cut stale answers from 65.8% to 41.9%, mostly through the classifier, without reducing recall on LoCoMo. H1 is only partly supported, because TSOM was the least accurate system on our change scenarios, where the classifier retired values that should have been kept. H2 is not supported, and the first part of H3 holds, since the fact layer roughly triples the LLM calls per write.

Most remaining stale answers occur with the correct value already marked [CURRENT], so the model's own knowledge, not the memory, is now the main bottleneck, followed by extraction. Future work should enable TSOM's multi-valued and "also" rules, resolve entity aliases, test larger models and a prompt without CURRENT markers, evaluate the retriever on dated data, run the llm_only classifier to test the rest of H3, and try a gate that skips A-Mem's evolution step when no conflict is found.

REFERENCES

[1] J. S. Park, J. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proc. 36th Annu. ACM Symp. User Interface Softw. Technol. (UIST)*, 2023, pp. 1–22, doi: https://doi.org/10.1145/3586183.3606763.

[2] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459–9474. [Online]. Available: https://arxiv.org/abs/2005.11401

[3] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang, "A-Mem: Agentic memory for LLM agents," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2025, doi: https://doi.org/10.52202/085713-0593.

[4] S. Bae *et al.*, "Keep me updated! Memory management in long-term conversations," in *Findings Assoc. Comput. Linguistics: EMNLP 2022*, 2022, pp. 3769–3787, doi: https://doi.org/10.18653/v1/2022.findings-emnlp.276.

[5] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, "Mem0: Building production-ready AI agents with scalable long-term memory," in *Proc. 28th Eur. Conf. Artif. Intell. (ECAI)*, ser. Frontiers in Artificial Intelligence and Applications, vol. 413, 2025, doi: https://doi.org/10.3233/FAIA251160.

[6] S. Yan *et al.*, "Memory-R1: Enhancing large language model agents to manage and utilize memories via reinforcement learning," in *Proc. 64th Annu. Meeting Assoc. Comput. Linguistics (Vol. 1: Long Papers)*, San Diego, CA, USA, 2026, pp. 12805–12825, doi: https://doi.org/10.18653/v1/2026.acl-long.583.

[7] P. Anokhin *et al.*, "AriGraph: Learning knowledge graph world models with episodic memory for LLM agents," in *Proc. 34th Int. Joint Conf. Artif. Intell. (IJCAI)*, 2025, pp. 12–20, doi: https://doi.org/10.24963/ijcai.2025/2.

[8] B. Jiménez Gutiérrez, Y. Shu, W. Qi, S. Zhou, and Y. Su, "From RAG to memory: Non-parametric continual learning for large language models," in *Proc. 42nd Int. Conf. Mach. Learn. (ICML)*, PMLR, vol. 267, 2025, pp. 21497–21515. [Online]. Available: https://arxiv.org/abs/2502.14802

[9] C. Packer *et al.*, "MemGPT: Towards LLMs as operating systems," arXiv:2310.08560, 2023. [Online]. Available: https://arxiv.org/abs/2310.08560

[10] J. Ye *et al.*, "MemWeaver: Weaving hybrid memories for traceable long-horizon agentic reasoning," in *Findings Assoc. Comput. Linguistics: ACL 2026*, San Diego, CA, USA, 2026, pp. 12928–12956, doi: https://doi.org/10.18653/v1/2026.findings-acl.630.

Case	Writes	Context given to the model	Answer
S1: What position does Nick Rimando play?	goalkeeper (221, ADD); flanker (2153, REPLACE, retires goalkeeper)	[CURRENT] (Nick Rimando, position, flanker), ranked first	flanker (correct)
S2: Where does Ali live now?	Karachi (1, ADD); Lahore (2, REPLACE)	[CURRENT] (Ali, located_in, Lahore)	Lahore (correct)
F1: Which sport is Salem-Keizer Volcanoes associated with?	baseball (1257, ADD); basketball (1722, REPLACE)	[CURRENT] (Salem-Keizer Volcanoes, associated_with, basketball), ranked first	baseball (stale)
F2: What is the capital of United Kingdom?	London (533) under United Kingdom of Great Britain and Ireland; Rupnagar (819, ADD) under United Kingdom	[CURRENT] (Great Britain, capital_of, London) and [CURRENT] (United Kingdom, capital_of, Rupnagar)	London (stale)

TABLE 10
AGENT TRAJECTORIES FOR TWO SUCCESSES (S) AND TWO FAILURES (F), WITH WRITES AS (STEP, OPERATION). EACH CASE USED ONE SEARCH_MEMORY CALL; S2 IS A CHANGE SCENARIO, THE REST ARE FACTCONSOLIDATION SH 32K (SEED 1).

[11] J. Zhu, J. Li, C. Zhang, J. Liu, and M. Yang, "HeLa-Mem: Hebbian learning and associative memory for LLM agents," in *Proc. 64th Annu. Meeting Assoc. Comput. Linguistics (Vol. 1: Long Papers)*, San Diego, CA, USA, 2026, pp. 13757–13769, doi: https://doi.org/10.18653/v1/2026.acl-long.625.

[12] J. Fang *et al.*, "LightMem: Lightweight and efficient memory-augmented generation," in *Proc. 14th Int. Conf. Learn. Represent. (ICLR)*, 2026. [Online]. Available: https://arxiv.org/abs/2510.18866

[13] B. Dhingra, J. R. Cole, J. M. Eisenschlos, D. Gillick, J. Eisenstein, and W. W. Cohen, "Time-aware language models as temporal knowledge bases," *Trans. Assoc. Comput. Linguistics*, vol. 10, pp. 257–273, 2022, doi: https://doi.org/10.1162/tacl_a_00459.

[14] P. Banerjee, M. Moshtaghi, S. Subramanian, A. Misra, and A. Chadha, "APEX-MEM: Agentic semi-structured memory with temporal reasoning for long-term conversational AI," in *Proc. 64th Annu. Meeting Assoc. Comput. Linguistics (Vol. 1: Long Papers)*, San Diego, CA, USA, 2026, pp. 16470–16489, doi: https://doi.org/10.18653/v1/2026.acl-long.749.

[15] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef, "Zep: A temporal knowledge graph architecture for agent memory," arXiv:2501.13956, 2025. [Online]. Available: https://arxiv.org/abs/2501.13956

[16] Y. Hu, Y. Wang, and J. McAuley, "Evaluating memory in LLM agents via incremental multi-turn interactions," in *Proc. 14th Int. Conf. Learn. Represent. (ICLR)*, 2026. [Online]. Available: https://arxiv.org/abs/2507.05257

[17] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu, "LongMemEval: Benchmarking chat assistants on long-term interactive memory," in *Proc. 13th Int. Conf. Learn. Represent. (ICLR)*, 2025. [Online]. Available: https://arxiv.org/abs/2410.10813

[18] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang, "Evaluating very long-term conversational memory of LLM agents," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (Vol. 1: Long Papers)*, Bangkok, Thailand, 2024, pp. 13851–13870, doi: https://doi.org/10.18653/v1/2024.acl-long.747.

[19] A. Yang *et al.*, "Qwen2.5 technical report," arXiv:2412.15115, 2024. [Online]. Available: https://arxiv.org/abs/2412.15115

[20] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020. [Online]. Available: https://arxiv.org/abs/2002.10957

[21] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000+ questions for machine comprehension of text," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, Austin, TX, USA, 2016, pp. 2383–2392, doi: https://doi.org/10.18653/v1/D16-1264.

[22] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Boca Raton, FL, USA: Chapman and Hall/CRC, 1994, doi: https://doi.org/10.1201/9780429246593.

[23] Z. Zhong, Z. Wu, C. D. Manning, C. Potts, and D. Chen, "MQuAKE: Assessing knowledge editing in language models via multi-hop questions," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, Singapore, 2023, doi: https://doi.org/10.18653/v1/2023.emnlp-main.971.

[24] S. Longpre, K. Perisetla, A. Chen, N. Ramesh, C. DuBois, and S. Singh, "Entity-based knowledge conflicts in question answering," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, Punta Cana, Dominican Republic, 2021, doi: https://doi.org/10.18653/v1/2021.emnlp-main.565.

[25] R. Xu, Z. Qi, Z. Guo, C. Wang, H. Wang, Y. Zhang, and W. Xu, "Knowledge conflicts for LLMs: A survey," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, Miami, FL, USA, 2024, doi: https://doi.org/10.18653/v1/2024.emnlp-main.486.